

Revisão da Fase 3 — Catálogo de Produtos e Controle de Estoque

product-service (MongoDB) e inventory-service (PostgreSQL) — construção, integração, cache e testes

A Fase 3 entregou os dois serviços que formam o núcleo comercial da plataforma: o **product-service** (catálogo, em MongoDB) e o **inventory-service** (estoque, em PostgreSQL). Foi a primeira vez que o projeto usou MongoDB, a primeira integração síncrona real entre dois microserviços via REST, o primeiro uso de cache com Redis, e o maior bloco de testes automatizados até aqui. A fase foi dividida em 8 blocos, cada um endurecido por code review antes do merge. Ao todo, cerca de 15 Pull Requests e mais de 190 testes automatizados entre os dois serviços.

1. O que foi construído

A fase seguiu uma progressão deliberada: primeiro o catálogo completo (Blocos 1-3), depois o estoque completo (Blocos 4-5), então a integração entre eles (Bloco 6), o cache (Bloco 7) e por fim a suíte de testes e esta documentação (Bloco 8).

product-service — catálogo (MongoDB via Mongoose)

- CRUD completo de produtos com autorização por papel (ADMIN/SELLER escrevem, leitura pública).
- Soft delete: produtos não são apagados, apenas marcados como **active: false** — somem da listagem e do detalhe.
- Allowlist de campos no update (**pickAllowedFields**): impede alterar campos sensíveis como **active** via corpo cru.
- Busca por texto (índice de texto do Mongo), filtro por categoria e paginação validada (page/limit/total/totalPages).
- Validação de entrada estrita: preço negativo, page/limit inválidos e search fora do tamanho retornam 400 (não falham em silêncio).

inventory-service — estoque (PostgreSQL via Prisma)

- Modelo com **quantity** (total físico) e **reserved** (reservado em compras em andamento); disponível = quantity – reserved.
- Constraints CHECK no banco: quantity ≥ 0, reserved ≥ 0, reserved ≤ quantity — impedem estado de estoque impossível mesmo com bug na aplicação.
- Reserva **atômica** via UPDATE condicional em SQL bruto (\$executeRaw): fecha a fresta de concorrência entre ler e escrever.
- Operações: setStock, getAvailability, reserveStock e releaseStock, com validação numérica e release restrito a ADMIN/SELLER.

Integração e cache

- product-service consulta o inventory-service via REST (fetch nativo) para enriquecer o detalhe do produto com disponibilidade.
- Comunicação resiliente: timeout com AbortController, degradação graciosa (inventory fora → produto ainda aparece, sem disponibilidade).

- Cache-aside no Redis para as listagens, com invalidação por versão (INCR) e TTL de 60s.

2. Percurso bloco a bloco

Bloco 1 — Estrutura base do product-service (PR #9)

Inicialização com Express + TypeScript + Mongoose, conexão ao MongoDB, health check na porta 3003. Primeiro contato com um ODM (Object Document Mapper) — o equivalente do Prisma para MongoDB. Diferente do SQL, não há migrations: o Mongo cria a collection na primeira inserção.

Bloco 2 — Modelo e CRUD de produtos (PR #10)

Schema do produto (name, description, price com mínimo 0, category indexada, imageUrl, active). CRUD com rotas de escrita protegidas por papel. O review pegou cinco falhas de segurança reais: validação do Mongoose que não rodava no update (faltava runValidators), soft delete vazando na listagem, e corpo cru permitindo reativar produto deletado. Todas corrigidas antes do merge.

Bloco 3 — Categorias, busca e paginação (PR #11)

Busca por texto, filtro por categoria e paginação. Discussão de performance: listar tudo de uma vez não escala. O review reforçou uma lição central — falhar em silêncio esconde erros do cliente. page/limit/search inválidos passaram a retornar 400 em vez de corrigir silenciosamente. Índice composto (active + createdAt) para a listagem.

Bloco 4 — Estrutura base do inventory-service (PR #12)

Segundo serviço, de volta ao PostgreSQL com Prisma — escolhido pela consistência transacional que o estoque exige. O review trouxe oito pontos; os mais valiosos foram as constraints CHECK (coração da consistência) e a ordem dotenv/Prisma (o PrismaClient era instanciado antes do env carregar). Também a separação app.ts/server.ts, que seria pré-requisito dos testes do Bloco 8.

Bloco 5 — Lógica de estoque (PR #14)

setStock, getAvailability, reserveStock e releaseStock. O bloco mais difícil da fase: a primeira implementação da reserva usava uma transação que NÃO travava a linha — a fresta de concorrência continuava aberta. O review pegou o que os testes sequenciais não pegavam. A correção foi um UPDATE condicional atômico em SQL bruto. Ficou a lição: concorrência não se valida com teste sequencial.

Bloco 6 — Integração product ↔ inventory (PRs #16-#18)

Primeira comunicação síncrona entre microserviços do projeto. O product-service consulta o inventory via REST. Decisões de projeto: available é RECALCULADO a partir de quantity – reserved (nunca se confia no número recebido), exposição reduzida (só available + inStock), timeout com AbortController, degradação graciosa e encoding do productId na URL. Aqui nasceu o docs/TECH_DEBT.md.

Bloco 7 — Cache com Redis (PR #19)

Cache-aside nas listagens. O bloco com mais idas e vindas de review — e o que mais ensinou sobre os problemas clássicos de cache: a corrida de invalidação (resolvida com versão via INCR, não varrendo o keyspace com keys()), a colisão de chave (resolvida com serialização não-ambígua + hash SHA-1), e a inconsistência de tipos hit vs miss (resolvida com .lean()). Decisão consciente: aceitar stale de até 60s via TTL e adiar circuit breaker/single-flight.

Bloco 8 — Testes e documentação (PRs #22-#27)

O fechamento da fase. Testes dos dois serviços, divididos em sub-blocos: 8a (inventory service/estrutura/env), 8b (product HTTP), 8c-1 (inventory.client), 8c-2 (rotas HTTP do inventory) e 8c-3 (findProductById + cache). Incluiu o teste de concorrência obrigatório — duas reservas simultâneas do último item, provando o lock atômico do Bloco 5. Resultado: product-service com 10 suítes/87 testes e inventory-service com 6 suítes/54 testes.

3. Conceitos técnicos consolidados

Banco de documentos vs. relacional. MongoDB no catálogo (flexibilidade de schema para produtos com atributos variados por categoria); PostgreSQL no estoque (consistência transacional ACID para evitar que duas pessoas comprem o último item).

ODM vs. ORM. Mongoose (ODM) define schemas em TypeScript e valida documentos, sem migrations. Prisma (ORM) usa schema .prisma e migrations versionadas. Cada banco no seu ponto forte.

Constraints no nível do banco. CHECK constraints garantem invariantes ($\text{quantity} \geq 0$, $\text{reserved} \leq \text{quantity}$) mesmo que um bug passe pela aplicação. É a última linha de defesa contra estado impossível.

Condição de corrida e reserva atômica. Ler-depois-escrever em duas etapas abre uma fresta sob concorrência. A solução é uma única instrução atômica: `UPDATE ... SET reserved = reserved + amount WHERE quantity - reserved >= amount`.

Soft delete. Em vez de apagar, marca-se `active: false`. Preserva histórico e integridade referencial. Exige lembrar do filtro `active: true` em toda query de leitura.

Allowlist de campos. No update, só campos de uma lista fixa passam. Impede que um corpo cru altere campos sensíveis (`active`). Mesmo princípio da allowlist de nomes de erro no log.

Comunicação síncrona resiliente. REST entre serviços com timeout (`AbortController`) e degradação graciosa: a falha de um serviço não derruba o outro. O cliente recalcula valores derivados em vez de confiar no recebido.

Cache-aside e invalidação por versão. A aplicação consulta o cache; no miss, busca no banco e grava. A invalidação incrementa uma versão (INCR) que entra na chave — órfã o cache antigo em $O(1)$, sem varrer o keyspace.

Colisão de chave de cache. Concatenar parâmetros com delimitador pode colidir (`category='a:b'` vs `search='b'`). Serializar como array JSON antes do hash SHA-1 torna a colisão estruturalmente impossível.

Consistência de tipos hit vs. miss. `.lean()` no Mongoose devolve objeto puro, igual ao que sai do `JSON.parse` do cache — garante que hit e miss retornem o mesmo formato.

Isolamento de banco em teste. Dois padrões: PostgreSQL real com banco dedicado + salvaguarda (`inventory`) e MongoDB em memória com `mongodb-memory-server` (`product`). O segundo isola por construção; o primeiro exige Docker ativo.

Testar o mecanismo, não só o efeito. Um teste de cache-hit deve provar que o banco NÃO foi consultado (spy em `find/countDocuments`), não só que o valor voltou. Um teste de timeout deve provar que o `AbortSignal` é configurado, não só que um erro vira null.

Mock funcional vs. instrumentado vs. stateful. Mock no-op silencia ruído; instrumentado (`jest.fn`) permite inspecionar chamadas; stateful (store compartilhado) liga leitura e escrita — necessário para provar que ler a versão após um INCR reflete o incremento.

Asserção forte vs. fraca. `not.toBe(401)` passa para qualquer coisa que não seja 401 (até 500). Afirmar o estado positivo esperado (`toBe(201)`) prova mais. `typeof === 'object'` aceita null; verificar a forma do erro é mais preciso.

process.exit só no entry point. Lógica testável extrai validação para funções puras que lançam erros; o process.exit fica só no server.ts. Permite testar sem matar o test runner.

4. Comandos e ferramentas aprendidos

Testes (Iest)

```
npx jest --ci # roda toda a suíte (reporter simples, sem barra animada)
npx jest product.cache --ci # roda só uma suíte pelo nome do arquivo
npm run verify # build (tsconfig.build.json) + test – prova que produção compila
npx jest --listTests # lista os arquivos de teste descobertos
```

Prisma / PostareSQL

```
npx prisma migrate dev --name init # cria e aplica migration
npx prisma generate # gera o Prisma Client
docker compose ps # confirma se os bancos estão Up (obrigatório p/ testes do inventory)
```

Git (fluxo de PR)

```
git checkout main && git pull # SEMPRE antes de criar branch (evita branch defasada)
git checkout -b feature/bloco-X-nome
git config pull.ff only # evita ambiguidade de reconciliação
git check-ignore .env.test # confirma que um arquivo está ignorado
git branch -d feature/bloco-X-nome # limpa a branch após o merge
```

Ferramentas da fase

- Jest + ts-jest + Supertest — runner, suporte a TypeScript e requisições HTTP em memória.
- mongodb-memory-server — MongoDB efêmero em memória para testes do product-service.
- fake timers do Jest — testar timeout/abort sem esperar tempo real.
- helpers de teste versionados — geração de JWT real, mock de Redis (funcional e instrumentado).
- docs/TECH_DEBT.md — registro vivo de toda dívida técnica, organizada por destino.

5. Fluxo de Pull Request e o valor do code review

Toda a fase seguiu o fluxo obrigatório: branch por bloco, PR, análise antes do merge, e limpeza da branch depois. O code review não foi formalidade — pegou problemas reais que os testes manuais não pegavam. A tabela resume os PRs.

PR	Bloco	Resultado do review
#9	1 — estrutura product	Aprovado; base sólida com MongoDB
#10	2 — CRUD produtos	5 falhas de segurança corrigidas (runValidators, soft delete, allowlist)
#11	3 — busca/paginação	Validação 400 em vez de falha silenciosa; índice composto
#12	4 — estrutura inventory	Constraints CHECK, ordem dotenv/Prisma, app/server separados
#14	5 — lógica estoque	Reserva atômica (\$executeRaw) — corrigiu corrida que a transação não fechava
#16-18	6 — integração REST	available recalculado, degradação graciosa, TECH_DEBT.md nasce
#19	7 — cache Redis	Invalidação por versão, chave SHA-1, .lean(); review mais denso
#22/#24	8a — testes inventory	30 testes + sanitizeConnectionError (allowlist, não confia em error.name)
#25	8b — testes product	51 testes; fix returnDocument:after; verify=build+test
#26	8c-1 — inventory.client	15 testes; timeout real (AbortSignal + fake timers)
#27	8c-3 — findById + cache	20 testes; mock stateful; hit prova que não consulta o banco
(merge)	8c-2 — rotas inventory	21 testes; reserve por papel (it.each); validação de estado

A lição transversal: compilar e passar em testes manuais não prova correção. O review pegou a corrida de concorrência do Bloco 5, as asserções fracas que passavam por sorte, o mock que não ligava leitura à escrita, e a documentação que prometia mais do que os testes provavam. Cada rodada tornou o código mais honesto.

6. Histórico de problemas e soluções

Cada erro enfrentado na fase, com causa e solução. É o registro mais valioso do documento — os erros viram prevenção nas próximas fases.

Erro: Validação do Mongoose não rodava no update (review PR #10)

Causa: findOneAndUpdate por padrão não roda os validators do schema.

Por que aconteceu: Um update com preço negativo passaria, driblando o min: 0 do schema.

Solução: Adicionar runValidators: true nas opções do update.

Erro: Soft delete vazando na listagem (review PR #10)

Causa: As queries de leitura não filtravam por active: true.

Por que aconteceu: Produtos marcados como inativos continuavam aparecendo na listagem e no detalhe.

Solução: Adicionar active: true no filtro de findAll e findById.

Erro: Corpo cru permitia reativar produto deletado (review PR #10)

Causa: O update aceitava o req.body inteiro, incluindo o campo active.

Por que aconteceu: Um cliente podia enviar active: true e ressuscitar um produto soft-deleted.

Solução: Allowlist de campos (pickAllowedFields): só name, description, price, category, imageUrl passam.

Erro: Paginação falhava em silêncio (review PR #11)

Causa: page/limit inválidos eram corrigidos com Math.max em vez de rejeitados.

Por que aconteceu: limit=abc por bug do frontend devolvia 20 itens como se nada fosse, escondendo o erro.

Solução: Validar e retornar 400 para page/limit/search inválidos (parsePositiveInt).

Erro: Ordem dotenv/Prisma (review PR #12)

Causa: O PrismaClient era instanciado no import do database.ts, antes do dotenv.config() do server.ts.

Por que aconteceu: Em outro ambiente, o Prisma poderia não achar a DATABASE_URL. Funcionava por sorte no ts-node-dev.

Solução: Carregar o dotenv dentro do database.ts, antes de criar o PrismaClient.

Erro: Banco aceitava estoque impossível (review PR #12)

Causa: Sem constraints, o Postgres aceitava quantity negativo e reserved > quantity.

Por que aconteceu: Um bug na aplicação poderia gravar um estado de estoque que não existe na vida real.

Solução: Constraints CHECK: quantity >= 0, reserved >= 0, reserved <= quantity.

Erro: Importar o app subia o servidor (review PR #12)

Causa: server.ts chamava startServer() e exportava o app no mesmo arquivo.

Por que aconteceu: Um teste que importasse o app subiria o servidor e conectaria ao banco de verdade.

Solução: Separar app.ts (cria o app) de server.ts (valida env, conecta e escuta).

Erro: A transação não travava a linha (review PR #14) — o erro mais grave da fase

Causa: A \$transaction fazia findUnique + update, mas sem SELECT FOR UPDATE nem isolamento serializável.

Por que aconteceu: A fresta de concorrência continuava aberta: duas reservas simultâneas liam o mesmo disponível. Testes sequenciais não pegavam.

Solução: UPDATE condicional atômico em SQL bruto (\$executeRaw): verificação e escrita numa única instrução indivisível.

Erro: setStock podia reduzir quantity abaixo do reserved (review PR #14)

Causa: setStock definia quantity sem checar o reserved atual.

Por que aconteceu: Definir quantity=3 com 5 reservados deixaria o estoque inconsistente (ou 500 pela constraint).

Solução: Guard QUANTITY_BELOW_RESERVED antes do upsert.

Erro: Corrida de invalidação de cache (review PR #19)

Causa: A invalidação inicial dependia de varrer chaves; sob concorrência, um cache stale podia sobreviver.

Por que aconteceu: Entre gravar o cache e invalidá-lo, uma escrita concorrente podia deixar dado velho servindo.

Solução: Invalidação por VERSÃO: um INCR muda a versão que entra na chave, orfanando o cache antigo em O(1).

Erro: Colisão de chave de cache (review PR #19)

Causa: Parâmetros concatenados com delimitador podiam gerar a mesma string.

Por que aconteceu: category='a:s=b' e search='b' poderiam colidir na mesma chave antes do hash.

Solução: Serializar como array JSON (JSON.stringify([...])) antes do SHA-1 — colisão estruturalmente impossível.

Erro: Tipos diferentes entre hit e miss (review PR #19)

Causa: O miss vinha como documento Mongoose; o hit, como objeto puro do JSON.parse.

Por que aconteceu: O consumidor recebia formatos ligeiramente diferentes dependendo de acertar ou não o cache.

Solução: Usar .lean() na query — devolve objeto puro, igual ao do cache.

Erro: error.name é mutável — não confiar nele (review follow-up PR #24)

Causa: sanitizeConnectionError logava error.name diretamente.

Por que aconteceu: name é uma propriedade mutável; um erro poderia carregar uma connection string nele e vaziar no log.

Solução: Allowlist de nomes conhecidos; qualquer outro cai numa categoria fixa. A saída nunca vem do erro.

Erro: Mongo em memória estourava timeout ao rodar em paralelo (Bloco 8b)

Causa: Cada suíte subia sua própria instância; 4 instâncias competindo estouravam o start de 10s.

Por que aconteceu: O Jest roda suítes em paralelo por padrão; várias instâncias do MongoMemoryServer brigavam por recursos.

Solução: maxWorkers: 1 no jest.config — roda em série, uma instância por vez.

Erro: Mock não ligava leitura da versão à escrita (review PR #27)

Causa: O mock de Redis devolvia valores fixos, sem store compartilhado entre get/set/incr.

Por que aconteceu: Se o código lesse uma chave de versão diferente da que incrementa, o teste passaria mesmo com o bug.

Solução: Mock STATEFUL: get/set/incr compartilham um Map real; ler após o INCR reflete o incremento.

Erro: 500 misterioso nos testes de rota do inventory (Bloco 8c-2)

Causa: Testes HTTP do inventory davam 500 ao tocar o banco.

Por que aconteceu: O Postgres do Docker estava desligado; diferente do Mongo em memória, o inventory usa banco real.

Solução: Docker/Postgres precisa estar Up para os testes de rota do inventory. Primeira verificação: docker compose ps.

Erro: Branch criada de uma main desatualizada (Bloco 6)

Causa: Uma branch nova nasceu sem 6 commits que já estavam no origin/main.

Por que aconteceu: Faltou git pull antes de criar a branch; ela nasceu defasada, sem o inventory.client.ts.

Solução: Sempre git pull antes de criar branch. git config pull.ff only previne ambiguidade de reconciliação.

7. Checklist de conclusão da fase

[OK] Funcionando

Os dois serviços sobem e respondem; endpoints testados manualmente e via curl.

[OK] Testado

product-service: 10 suítes / 87 testes. inventory-service: 6 suítes / 54 testes. Todos passando, incluindo o teste de concorrência.

[OK] Commitado

~15 PRs mergeados no padrão Conventional Commits, cada um revisado antes do merge.

[OK] Documentado

README atualizado, docs/TECH_DEBT.md completo (dívida por destino) e este PDF de revisão.

[OK] Compreendido

Conceitos validados ao longo dos blocos: documentos vs. relacional, reserva atômica, cache-aside, padrões de teste.

[OK] Revisado

Cada bloco passou por code review; várias correções de segurança e concorrência antes do merge.

[OK] Seguro

Autorização por papel, allowlist de campos, constraints de estoque, log sanitizado, .env e .env.test bloqueados.

8. Dívida técnica e próximos passos

Nada foi varrido para baixo do tapete: cada adiamento consciente está em docs/TECH_DEBT.md, organizado pelo destino onde será resolvido. Resumo das principais pendências:

Fase 4 — order-service (próxima)

- Carrinho e pedidos. Resolve o ownership real da reserva: hoje release é restrito a ADMIN/SELLER como mitigação, mas não valida de quem é a reserva — o order-service amarra cada reserva a um pedido/usuário.

Fase 7 — API Gateway e segurança

- JWT hardening (issuer/audience/algorithms explícitos + validação do shape do payload).
- 403 genérico que não vaza roles; autenticação serviço-a-serviço (token interno/mTLS) entre product e inventory.
- Separar /health (liveness) de /ready (readiness).

Fase 7/10 — infra, resiliência e CI

- Timeout configurável + circuit breaker/retry no inventory.client; cache stampede/single-flight; logger estruturado.

- CI (Fase 10): setar JWT_SECRET e env de teste no pipeline; rodar npm run verify; fixar versão do Mongo em memória + cache do binário.
- Testes diferidos: config do redis.ts (REDIS_URL ausente) e integração do connectDatabase.

Refatoração dedicada

- Erros de domínio como enums (não strings); estender o tipo Request do Express; DTO ProductWithAvailability; helpers puros para módulo utils/.

Herdadas da Fase 2

- Testes e2e com Supertest (incluindo o 429 do rate limit); paginação em /admin/users; login social via OAuth2.

Fase 3 concluída | product-service e inventory-service completos, integrados, com cache e testados | Davi
x Claude